

中国矿业大学计算机学院

23 级本科生课程作业 2

课程名称 Linux 操作系统

作业内容 Linux shell 编程

学生姓名 王志豪

学 号 08232337

专 业 数据科学与大数据技术

任课教师 李昕

LINUX 下的 shell 编程案例

一、源代码展示：

```
1  #!/bin/bash
2
3  # 设置日志目录
4  LOG_DIR="/var/log/smart_monitor"
5  REPORT_FILE="$LOG_DIR/daily_report.log"
6  CONFIG_FILE="/etc/smart_monitor.conf"
7  TMP_DIR="/tmp/smart_monitor"
8  MAX_LOG_SIZE=10240 # 单个日志最大10KB
9  #默认配置
10 CPU_THRESHOLD=80
11 MEM_THRESHOLD=80
12 DISK_THRESHOLD=85
13 NETWORK_INTERFACE="eth0"
14
15 mkdir -p "$LOG_DIR" "$TMP_DIR"
16
17 # 日志记录函数，将信息写入日志文件与终端
18 log() {
19     local msg="$1"
20     echo "$(date '+%F %T') : $msg" | tee -a "$REPORT_FILE"
21 }
22
23 line() {
24     echo "---" | tee -a "$REPORT_FILE"
25 }
26
27 # 彩色输出函数（便于区分信息类型）
28 color_echo() {
29     local color=$1
30     local text=$2
31     case $color in
32         red) echo -e "\033[31m$text\033[0m" ;;
33         green) echo -e "\033[32m$text\033[0m" ;;
34         yellow) echo -e "\033[33m$text\033[0m" ;;
35         blue) echo -e "\033[34m$text\033[0m" ;;
36         *) echo "$text" ;;
37     esac
38 }
39
40 # 检查日志文件大小，超过阈值则轮换（重命名备份）
41 rotate_log() {
42     if [ -f "$REPORT_FILE" ]; then
43         local size
44         size=$(stat -c%s "$REPORT_FILE") # 获取文件大小（字节）
45         if [ "$size" -gt "$MAX_LOG_SIZE" ]; then
```

图 1：源代码片段 1

```

46         mv "$REPORT_FILE" "$REPORT_FILE.${date +%s}.bak"
47         echo "" > "$REPORT_FILE"
48         log "日志已轮换。"         fi
49     fi
50 }
51
52 # 检查 CPU 使用率
53 check_cpu() {
54
55     local usage total_usage samples=5
56     total_usage=0
57
58     # 多层循环：进行多次 CPU 采样，提高准确率
59     for round in $(seq 1 $samples); do
60         # 第二层循环：进行微小延迟采样（模拟更精细的监控）
61         for sub in 1 2 3; do
62             sleep 0.1
63         done
64
65         if command -v mpstat >/dev/null 2>&1; then
66             # 使用 mpstat 采样 (Average idle)
67             current=$(mpstat 1 1 | awk '/Average/ {print 100 - $12}')
68         else
69             # 使用 top fallback
70             current=$(top -bn1 | grep -E "Cpu|%Cpu" | sed 's/,/ /g' \
71                 | awk '{for(i=1;i<=NF;i++){if($i~/id/){print 100-$i; exit}}}')
72         fi
73
74         # 若采样为空，跳过该次采样
75         if [ -z "$current" ]; then
76             continue
77         fi
78
79         # 累加
80         total_usage=$(echo "$total_usage + $current" | bc)
81     done
82
83     # 计算平均用量
84     usage=$(echo "$total_usage / $samples" | bc)
85     usage=${usage%.*}
86
87     log "CPU使用率: $usage%"

```

图 2：源代码片段 2

```

89     # 原有逻辑保持不变
90     if [ -z "$usage" ]; then
91         log "无法获取CPU使用率, 请检查系统命令。"
92         return 0
93     fi
94
95     if [ "$usage" -gt "$CPU_THRESHOLD" ]; then
96         log "CPU使用率过高: $usage%"
97         return 1
98     else
99         return 0
100    fi
101 }
102 # 检查内存使用率
103 check_mem() {
104     local usage
105     usage=$(free | awk '/Mem/{printf("%.0f"), $3/$2 * 100}')
106     log "内存使用率: $usage%"
107     if [ "$usage" -gt "$MEM_THRESHOLD" ]; then
108         log "内存使用率过高: $usage%"
109         return 1
110     else
111         return 0
112     fi
113 }
114
115 # 检查磁盘使用率（根目录）
116 check_mem() {
117     local usage
118
119     # 优先使用 free（最稳定）
120     if command -v free >/dev/null 2>&1; then
121         usage=$(free | awk '/Mem/{printf("%.0f"), $3/$2 * 100}')
122
123     # 其次使用 vmstat（部分精简系统没有 free）
124     elif command -v vmstat >/dev/null 2>&1; then
125         usage=$(vmstat -s | awk '
126             /total memory/ {total=$1}
127             /used memory/ {used=$1}
128             END {if (total > 0) printf("%.0f", used/total * 100); else print 0}
129             ')
130     else
131         log "无法获取内存使用率: 系统缺少 free 与 vmstat 命令。"
132         return 0

```

图 3：源代码片段 3

```

133     fi
134
135     # 去掉小数
136     usage=${usage%.*}
137
138     log "内存使用率: $usage%"
139
140     if [ -z "$usage" ]; then
141         log "内存使用率获取失败。"
142         return 0
143     fi
144
145     if [ "$usage" -gt "$MEM_THRESHOLD" ]; then
146         log "内存使用率过高: $usage%"
147         return 1 # 返回1表示异常
148     else
149         return 0
150     fi
151 }
152 check_network() {
153     local tx rx
154     tx=$(cat /sys/class/net/$NETWORK_INTERFACE/statistics/tx_bytes)
155     rx=$(cat /sys/class/net/$NETWORK_INTERFACE/statistics/rx_bytes)
156     sleep 1 # 等1秒计算速率
157     local tx_new rx_new
158     tx_new=$(cat /sys/class/net/$NETWORK_INTERFACE/statistics/tx_bytes)
159     rx_new=$(cat /sys/class/net/$NETWORK_INTERFACE/statistics/rx_bytes)
160     local tx_rate=$(( (tx_new - tx) / 1024 ))
161     local rx_rate=$(( (rx_new - rx) / 1024 ))
162     log "网络速率: 上行 ${tx_rate}KB/s, 下行 ${rx_rate}KB/s"
163 }
164
165 # 修复CPU高占用: 杀死高CPU进程
166 auto_fix_cpu() {
167     color_echo yellow "尝试修复CPU高占用问题..."
168     ps -eo pid,comm,%cpu --sort=-%cpu | head -n 6 | tee "$TMP_DIR/high_cpu.txt"
169     culprit_pid=$(awk 'NR==2{print $1}' "$TMP_DIR/high_cpu.txt")
170     if [ -n "$culprit_pid" ]; then
171         kill -9 "$culprit_pid" && log "已终止高CPU进程: $culprit_pid"
172     fi
173 }
174
175 # 清理内存缓存

```

图 4: 源代码片段 4

```

176 auto_fix_mem() {
177     color_echo yellow "尝试清理内存缓存..."
178     sync; echo 3 > /proc/sys/vm/drop_caches
179     log "已清理系统缓存。"
180 }
181
182 # 清理临时文件与旧日志
183 auto_fix_disk() {
184     color_echo yellow "尝试清理磁盘临时文件..."
185     rm -rf /tmp/* # 删除临时目录
186     journalctl --vacuum-time=3d >/dev/null 2>&1 # 清理3天前日志
187     log "已清理临时与旧日志文件。"
188 }
189
190 monitor_system() {
191     line
192     log "开始系统监控..."
193     local cpu_ok mem_ok disk_ok
194
195     # 调用检查函数并获取返回状态
196     check_cpu; cpu_ok=$?
197     check_mem; mem_ok=$?
198     check_disk; disk_ok=$?
199     check_network
200
201     # 根据检测结果执行修复操作
202     if [ $cpu_ok -ne 0 ]; then
203         auto_fix_cpu
204     fi
205     if [ $mem_ok -ne 0 ]; then
206         auto_fix_mem
207     fi
208     if [ $disk_ok -ne 0 ]; then
209         auto_fix_disk
210     fi
211     log "当前系统检查完毕。"
212 }
213
214 generate_report() {
215     line
216     log "系统运行报告: "
217     echo "主机名: $(hostname)" | tee -a "$REPORT_FILE"
218     echo "系统版本: $(uname -r)" | tee -a "$REPORT_FILE"
219     echo "运行时间: $(uptime -p)" | tee -a "$REPORT_FILE"
220     echo "内存状态: " | tee -a "$REPORT_FILE"

```

图 5: 源代码片段 5

```

221     free -h | tee -a "$REPORT_FILE"
222     echo "磁盘状态:" | tee -a "$REPORT_FILE"
223     df -h | tee -a "$REPORT_FILE"
224     line
225 }
226
227 # 读取配置文件（如存在）
228 load_config() {
229     if [ -f "$CONFIG_FILE" ]; then
230         source "$CONFIG_FILE"
231         log "加载配置文件: $CONFIG_FILE"
232     else
233         log "未找到配置文件，使用默认配置。"
234     fi
235 }
236
237 # 生成或重置默认配置文件
238 edit_config() {
239     cat <<EOF > "$CONFIG_FILE"
240 CPU_THRESHOLD=80
241 MEM_THRESHOLD=80
242 DISK_THRESHOLD=85
243 NETWORK_INTERFACE=eth0
244 EOF
245     log "已创建默认配置文件。"
246 }
247
248 show_menu() {
249     clear
250     echo "====="
251     color_echo blue "SmartSysMonitor 智能系统监控"
252     echo "====="
253     echo "1) 立即运行系统监控"
254     echo "2) 查看最新报告"
255     echo "3) 手动生成报告"
256     echo "4) 编辑配置文件"
257     echo "5) 清理日志"
258     echo "6) 退出"
259     echo "====="
260 }
261
262 while true; do
263     show_menu
264     read -p "请选择操作 [1-6]: " choice

```

图 6: 源代码片段 6

```
267         rotate_log
268         load_config
269         monitor_system
270         read -p "按回车继续..."
271         ;;
272     2)
273         less "$REPORT_FILE"
274         ;;
275     3)
276         generate_report
277         read -p "按回车继续..."
278         ;;
279     4)
280         edit_config
281         read -p "按回车继续..."
282         ;;
283     5)
284         rm -f "$LOG_DIR"/*.bak
285         log "已清理历史日志。"
286         read -p "按回车继续..."
287         ;;
288     6)
289         color_echo green "退出 SmartSysMonitor"
290         exit 0
291         ;;
292     *)
293         color_echo red "无效选择，请重试。"
294         ;;
295 esac
296 done
```

图 7：源代码片段 7

二、功能运行测试:

1.立即运行系统监控

运行 SysMonitor.sh 脚本文件即可打开选项面板，输入选项 1 即可立即运行系统监控，结果如下：

```
=====
SmartSysMonitor 智能系统监控
=====
1) 立即运行系统监控
2) 查看最新报告
3) 手动生成报告
4) 编辑配置文件
5) 清理日志
6) 退出
-----
请选择操作 [1-6]: 1
2025-11-10 14:57:54 : 加载配置文件: /etc/smart_monitor.conf
-----
2025-11-10 14:57:54 : 开始系统监控 ...
2025-11-10 14:57:54 : CPU使用率: 1%
2025-11-10 14:57:54 : 内存使用率: 32%
2025-11-10 14:57:54 : 磁盘使用率: 14%
cat: /sys/class/net/eth0/statistics/tx_bytes: No such file or directory
cat: /sys/class/net/eth0/statistics/rx_bytes: No such file or directory
cat: /sys/class/net/eth0/statistics/tx_bytes: No such file or directory
cat: /sys/class/net/eth0/statistics/rx_bytes: No such file or directory
2025-11-10 14:57:55 : 网络速率: 上行 0KB/s, 下行 0KB/s
2025-11-10 14:57:55 : 当前系统检查完毕。
按回车继续 ...
```

图 8: 功能 1 运行结果

模拟 CPU 高负载的情况，我们尝试启动两个占用 CPU 100%的进程：

```
[jaydon@localhost dir3]$ yes > /dev/null &
[1] 45155
[jaydon@localhost dir3]$ yes > /dev/null &
[2] 45545
```

图 9: 手动添加高占用进程

```
=====
1) 立即运行系统监控
2) 查看最新报告
3) 手动生成报告
4) 编辑配置文件
5) 清理日志
6) 退出
-----
请选择操作 [1-6]: 1
2025-11-10 15:19:45 : 加载配置文件: /etc/smart_monitor.conf
-----
2025-11-10 15:19:45 : 开始系统监控...
2025-11-10 15:19:46 : CPU使用率: 100%
2025-11-10 15:19:46 : ▲ CPU使用率过高: 100%
2025-11-10 15:19:46 : 内存使用率: 34%
2025-11-10 15:19:46 : 磁盘使用率: 14%
cat: /sys/class/net/eth0/statistics/tx_bytes: No such file or directory
cat: /sys/class/net/eth0/statistics/rx_bytes: No such file or directory
cat: /sys/class/net/eth0/statistics/tx_bytes: No such file or directory
cat: /sys/class/net/eth0/statistics/rx_bytes: No such file or directory
2025-11-10 15:19:47 : 网络速率: 上行 0KB/s, 下行 0KB/s
尝试修复CPU高占用问题...
  PID COMMAND          %CPU
 45155 yes              99.7
 45545 yes              90.5
  2822 gnome-shell      1.6
   139 rcuos/1          0.2
   970 rngd             0.2
2025-11-10 15:19:47 : 已终止高CPU进程: 45155
2025-11-10 15:19:47 : 当前系统检查完毕。
按回车继续...
```

图 10: 高负载占用自愈

通过运行结果可以发现 45155 进程被成功杀死，再次运行上述选项 1，即可杀死 45545 进程，进程占用优化问题得到解决

```
top - 15:21:22 up 23:22, 3 users, load average: 1.34, 1.13, 0.89
Tasks: 433 total, 1 running, 432 sleeping, 0 stopped, 0 zombie
%Cpu(s): 4.3 us, 1.0 sy, 0.0 ni, 94.7 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
KiB Mem: 3869044 total, 1313848 used, 2555196 free, 56 buffers
KiB Swap: 4079612 total, 0 used, 4079612 free. 166200 cached Mem

  PID USER      PR  NI   VIRT   RES   SHR S  %CPU  %MEM    TIME+  COMMAND
 2822 jaydon    20   0 2239008 688948 41292 S  12.6  17.8   23:13.98 gnome-shell
 1070 root      20   0 231144   60160 8376 S   2.0   1.6    2:08.98 Xorg
 3142 jaydon    20   0 644336   28024 15384 S   0.7   0.7    0:19.80 gnome-terminal-
  137 root      20   0      0      0      0 S   0.3   0.0    2:16.60 rcu_sched
  657 root      20   0      0      0      0 S   0.3   0.0    0:14.37 xfsaild/dm-1
   960 root      20   0 388760   11384 7136 S   0.3   0.3    0:05.17 NetworkManager
 2922 jaydon    20   0 1143912 34648 18104 S   0.3   0.9    0:15.09 nautilus
    1 root      20   0  53764    7664 2564 S   0.0   0.2    0:06.94 systemd
```

图 11: 优化后进程占用情况

2.查看最新报告

输入选项 2，可以查看最新的优化日志，可以从中发现有我们执行选项 1 的记录说明改功能正常运行。

```
2025-11-10 15:08:02 : 日志已轮换。
2025-11-10 15:08:02 : 加载配置文件: /etc/smart_monitor.conf
-----
2025-11-10 15:08:02 : 开始系统监控...
2025-11-10 15:08:02 : CPU使用率: 1%
2025-11-10 15:08:02 : 内存使用率: 32%
2025-11-10 15:08:02 : 磁盘使用率: 14%
2025-11-10 15:08:04 : 网络速率: 上行 0KB/s, 下行 0KB/s
2025-11-10 15:08:04 : 当前系统检查完毕。
2025-11-10 15:14:33 : 加载配置文件: /etc/smart_monitor.conf
-----
2025-11-10 15:14:33 : 开始系统监控...
2025-11-10 15:14:33 : CPU使用率: 1%
2025-11-10 15:14:33 : 内存使用率: 34%
2025-11-10 15:14:33 : 磁盘使用率: 14%
2025-11-10 15:14:34 : 网络速率: 上行 0KB/s, 下行 0KB/s
2025-11-10 15:14:34 : 当前系统检查完毕。
2025-11-10 15:14:47 : 加载配置文件: /etc/smart_monitor.conf
-----
2025-11-10 15:14:47 : 开始系统监控...
2025-11-10 15:14:47 : CPU使用率: 1%
2025-11-10 15:14:47 : 内存使用率: 34%
2025-11-10 15:14:47 : 磁盘使用率: 14%
2025-11-10 15:14:48 : 网络速率: 上行 0KB/s, 下行 0KB/s
2025-11-10 15:14:48 : 当前系统检查完毕。
2025-11-10 15:15:16 : 加载配置文件: /etc/smart_monitor.conf
-----
2025-11-10 15:15:16 : 开始系统监控...
2025-11-10 15:15:16 : CPU使用率: 1%
/var/log/smart_monitor/daily_report.log
```

图 12: 运行功能 2 结果

3. 手动生成报告

输入选项 3, 即可生成一份综合报告:

```

1) 立即运行系统监控
2) 查看最新报告
3) 手动生成报告
4) 编辑配置文件
5) 清理日志
6) 退出
-----
请选择操作 [1-6]: 3
-----
2025-11-10 15:26:48 : 系统运行报告 :
主机名: localhost.localdomain
系统版本: 3.10.0-123.el7.x86_64
运行时间: up 23 hours, 28 minutes
内存状态:
          total      used      free      shared    buffers    cached
Mem:      3.7G       1.3G       2.4G        10M        56K       162M
-/+ buffers/cache:      1.1G       2.6G
Swap:      3.9G         0B       3.9G

磁盘状态:
Filesystem      Size  Used Avail Use% Mounted on
/dev/mapper/centos-root 38G  5.0G   33G  14% /
devtmpfs         1.9G     0   1.9G   0% /dev
tmpfs            1.9G  148K   1.9G   1% /dev/shm
tmpfs            1.9G   8.9M   1.9G   1% /run
tmpfs            1.9G     0   1.9G   0% /sys/fs/cgroup
/dev/mapper/centos-home 19G   75M   19G   1% /home
/dev/sda1        497M  119M  379M  24% /boot
/dev/sr0         11G   11G     0 100% /run/media/jaydon/CentOS-8-5-2111-x86_64-dvd
-----
按回车继续 ...

```

图 13: 运行功能 3 结果

4. 编辑配置文件

输入选项 4，即可自动编写一份配置信息写入配置文件中，默认的配置信息如下：

CPU_THRESHOLD=80

MEM_THRESHOLD=80

DISK_THRESHOLD=85

NETWORK_INTERFACE=eth0

系统运行结果如下：

```
=====
SmartSysMonitor 智能系统监控
=====
```

- ```
1) 立即运行系统监控
2) 查看最新报告
3) 手动生成报告
4) 编辑配置文件
5) 清理日志
6) 退出
```

```

请选择操作 [1-6]: 4
```

```
2025-11-10 15:28:43 : 已创建默认配置文件。
```

```
按回车继续...
```

图 14: 运行功能 4 结果

如果我们手动更改 CPU\_THRESHOLD 的阈值, 例如更改为 5, 那么 CPU 占用超过 5%, 执行选项 1 系统也会触发自愈程序

```
CPU_THRESHOLD=5
MEM_THRESHOLD=80
DISK_THRESHOLD=85
NETWORK_INTERFACE=eth0
```

图 15: 手动配置阈值信息

执行结果如下, 发现自愈程序成功触发。

```

=====
1) 立即运行系统监控
2) 查看最新报告
3) 手动生成报告
4) 编辑配置文件
5) 清理日志
6) 退出

请选择操作 [1-6]: 1
2025-11-10 15:33:17 : 加载配置文件: /etc/smart_monitor.conf

2025-11-10 15:33:17 : 开始系统监控...
2025-11-10 15:33:18 : CPU使用率: 8%
2025-11-10 15:33:18 : ▲ CPU使用率过高: 8%
2025-11-10 15:33:18 : 内存使用率: 34%
2025-11-10 15:33:18 : 磁盘使用率: 14%
cat: /sys/class/net/eth0/statistics/tx_bytes: No such file or directory
cat: /sys/class/net/eth0/statistics/rx_bytes: No such file or directory
cat: /sys/class/net/eth0/statistics/tx_bytes: No such file or directory
cat: /sys/class/net/eth0/statistics/rx_bytes: No such file or directory
2025-11-10 15:33:19 : 网络速率: 上行 0KB/s, 下行 0KB/s
尝试修复CPU高占用问题...
 PID COMMAND %CPU
 2822 gnome-shell 1.6
 139 rcuos/1 0.2
 970 rngd 0.2
 45704 top 0.2
 137 rcu_sched 0.1
2025-11-10 15:33:19 : 已终止高CPU进程: 2822
2025-11-10 15:33:19 : 当前系统检查完毕。
按回车继续...

```

图 16: 调整阈值后功能 1 运行结果

## 5. 清理日志

检查 log 中存在原来的旧日志文件

```

[root@localhost ~]# ls -lh /var/log/smart_monitor
total 24K
-rw-r--r--. 1 root root 8.8K Nov 10 15:33 daily_report.log
-rw-r--r--. 1 root root 11K Nov 10 15:06 daily_report.log.1762758482.bak

```

图 17: 检查原有旧日志文件

现在输入选项 5

```

=====
SmartSysMonitor 智能系统监控
=====

1) 立即运行系统监控
2) 查看最新报告
3) 手动生成报告
4) 编辑配置文件
5) 清理日志
6) 退出

请选择操作 [1-6]: 5
2025-11-10 15:37:32 : 已清理历史日志。
按回车继续 ...

[root@localhost ~]# ls -lh /var/log/smart_monitor
total 12K
-rw-r--r--. 1 root root 8.9K Nov 10 15:37 daily_report.log
[root@localhost ~]#

```

图 17: 运行功能 5 的结果

发现旧日志文件被成功删除。

### 三、调试信息:

#### 1. 语法错误检查

执行 `bash -n` 检查是否有错，未发现语法错误

```

[root@localhost ~]# vi ./SysMonitor.sh
[root@localhost ~]# _bash -n ./SysMonitor.sh

```

图 18: 检查语法错误

#### 2. 功能检查

检查功能完备性，发现 CPU 检测并不够“实时”，很多时候存在高占用 CPU 进程但是检测不出，排查发现以下问题：

```
usage=$(top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8}')
```

该语句在不同系统语言环境下经常解析错误，因为 `top` 的输出格式在不同语言下差异很大。所以更换使用 `mpstat`（`sysstat` 包自带）这样统计最准确，

而 `top` 只当作备选方案。至此功能完备。

#### 四、个人体会：

通过完成本次 Linux Shell 编程作业，我深刻体会到了 Shell 脚本在实际系统管理中的强大作用。在开发系统监控脚本的过程中，我不仅巩固了 Shell 编程的基本语法，更重要的是学会了如何将理论知识转化为解决实际问题的能力。特别是在处理 CPU 使用率检测时，遇到了不同系统环境下输出格式差异的挑战，这让我意识到编写健壮的程序需要考虑各种边界情况。通过对比 `mpstat` 和 `top` 命令的差异，我学会了选择更可靠的工具来完成任务。整个调试过程培养了我的问题排查能力和耐心，当看到脚本成功识别并终止高 CPU 进程时，那种成就是无法言喻的。这次作业让我明白，优秀的系统管理员不仅要会使用命令，更要懂得如何通过编程让系统管理自动化、智能化。